Práctica 5

Manuel Suano Gallardo

Ejercicio 3

Algunas de las capturas de este ejercicio se encuentran en el Anexo debido al tamaño de estas.

Apartado 1.

Al ejecutar las comprobaciones se indica que el uso de CALL sin indicar un ámbito de variable está obsoleto, por lo que es necesario cambiar las consultas de CALL { … } a CALL () { … }. El resultado está en la sección Ejercicio 3.1.

Apartado 2.

Para cada obtener la información solicitada en cada una de las consultas es necesario ejecutar las consultas:

1. `MATCH (c:Club) ORDER BY c.nombre ASC RETURN c.nombre AS Club, c.ciudad AS Ciudad;`

En este caso obtenemos los nodos con la etiqueta Club y mostramos la información requerida.



2. `MATCH (c:Club)-[:JUEGA_EN_ESTADIO]->(e:Estadio) RETURN c.nombre, e.nombre;`

Usamos la relación JUEGA_EN_ESTADIO entre los nodos con etiqueta Club y Estadio para obtener el nombre del estadio de cada club.



3. `MATCH (e:Entrenador)-[:ENTRENA_A]->(c:Club) ORDER BY c.nombre DESC RETURN e.nombre, c.nombre;`

Caso parecido al anterior, usamos la relación ENTRENA_A para obtener el nombre de los entrenadores de cada club.



4. `MATCH (j:Jugador)-[:JUEGA_EN]->(c:Club) RETURN COUNT(j) as total, c.nombre ORDER BY total ASC;`

Aquí usamos la agrupación automática que hace Cypher para obtener el número de jugadores que tienen actualmente cada club. El número de jugadores lo obtenemos contando el número de nodos que tienen la relación JUEGA_EN con cada club.



5. `MATCH (p:Partido)-[:EQUIPO_LOCAL|EQUIPO_VISITANTE]->(c:Club) WHERE c.nombre='Albacete Balompie' RETURN p.partidoId;`

Como queremos todos los partidos, tanto los de local como visitante, indicamos que sean una de esas dos relaciones entre partido y club, además de indicar como nombre del club Albacete Balompie.



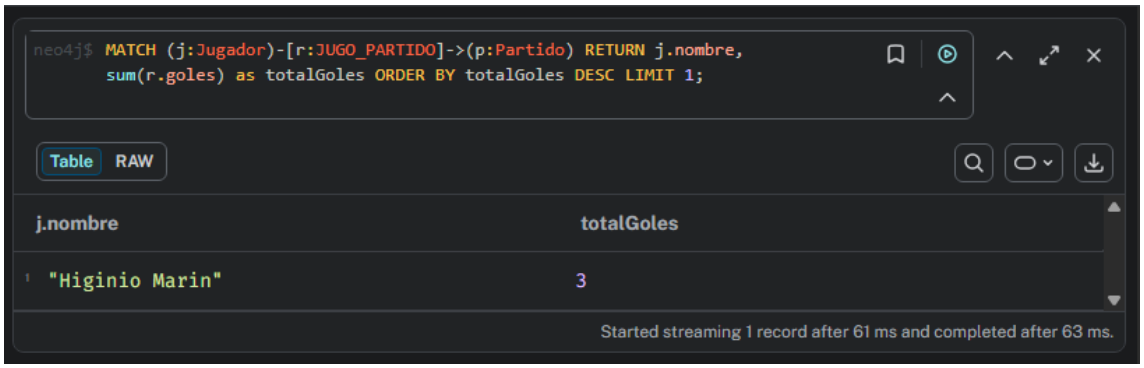
6. `MATCH (j:Jugador)-[:JUEGA_EN|JUGO_EN]->(c:Club) WITH j AS jugador, collect(DISTINCT c.nombre) AS clubes WHERE size(clubes) > 1 RETURN jugador.nombre, clubes ORDER BY size(clubes) DESC;`

Primero obtenemos los jugadores y los clubs donde juega y han jugado, sin distinción. La lista de clubes la agrupamos en un array y comprobamos que haya más de un elemento, lo que significa que ha estado en al menos un club, contando con el actual.



7. `MATCH (j:Jugador)-[r:JUGO_PARTIDO]->(p:Partido) RETURN j.nombre, sum(r.goles) as totalGoles ORDER BY totalGoles DESC LIMIT 1;`

A partir de la propiedad goles de la relación JUGO_PARTIDO, realizamos la suma de todos los goles marcados por un determinado jugador aprovechando el agrupamiento automático que realiza Cypher.



8. `MATCH (j:Jugador)-[r1:JUEGA_EN]->(c:Club) WITH j as jugador, c as clubActual MATCH (jugador)-[:JUGO_EN]->(clubActual) MATCH (jugador)-[:JUGO_EN|JUEGA_EN]->(c:Club) WITH jugador, collect(c.nombre) as historicoClubes RETURN jugador.nombre, historicoClubes;`

En este caso indicamos varias condiciones: primero obtenemos el club actual de cada jugador y después comprobamos que también exista la relación JUGO_EN entre ese jugador y club, para determinar que el jugador ha vuelto. Cuando se tiene esa información basta con obtener la lista de clubs con los que un jugador ha tenido contrato en algún momento y los mostramos junto con el nombre del jugador.



9. `MATCH (j:Jugador)-[r:JUGO_PARTIDO]->(p:Partido) MATCH (j)-[:JUGO_EN]->(exClub:Club) MATCH (p)-[:EQUIPO_LOCAL|EQUIPO_VISITANTE]->(exClub) MATCH (j)-[:JUEGA_EN]->(clubActual:Club) WHERE r.goles > 0 AND exClub <> clubActual RETURN DISTINCT j.nombre, exClub.nombre;`

Primero buscamos los jugadores que metieron al menos un gol en un partido (primer MATCH y condición r.goles > 0), después filtramos los partidos donde uno de los equipos es el exclub del jugador y que además no es el mismo club que el actual.



Apartado 3.

Un caso que se propone en la web de Neo4J[1] es el caso de detección de fraude. Para implementar una base de datos para este propósito definiremos 4 nodos:

- Cuenta. Cuyas propiedades será el IBAN y el balance.
- Persona. Con el DNI y el nombre.
- Medio. Este nodo representa la forma en que una persona realiza operación.

[1] https://go.neo4j.com/rs/710-RRC-335/images/Neo4j_Top5_UseCases_Graph%20Databases.pdf

- País. Donde está registrada la cuenta y si se trata de un paraíso fiscal según sus leyes.

Por otro lado, las relaciones existentes serán:

- Una persona es titular de una cuenta.
- Una cuenta transfiere tanto dinero en una determinada fecha a otra cuenta.
- En qué país está registrada una cuenta.
- En qué país vive una persona.
- Una persona usa un determinado medio.

Los datos que introduciremos para realizar las consultas serán:

```
MERGE (pe1:Persona {nombre: "Manuel Suano Gallardo", dni : "12345678A"})
MERGE (pe2:Persona {nombre: "Laura Muñoz Fernández", dni : "23456789B"})
MERGE (pe3:Persona {nombre: "José Sánchez García", dni : "34567890C"})
MERGE (c1:Cuenta {iban: "ES0100203122312342", balance: 1000000})
MERGE (c2:Cuenta {iban: "IC0548549080398509384", balance: 35922})
MERGE (c3:Cuenta {iban: "ES0109769078450988", balance: 5300})
MERGE (c4:Cuenta {iban: "RS6758678456348450988", balance: 30000})
MERGE (c5:Cuenta {iban: "ES0103487423874733", balance: 300})
MERGE (md1:Medio {valor: "86.98.23.111"})
MERGE (md2:Medio {valor: "124.2.53.6"})
MERGE (pa1:Pais {nombre: 'Islas Caimán', paraiso_fiscal: true})
MERGE (pa2:Pais {nombre: 'España', paraiso_fiscal: false})
MERGE (pa3:Pais {nombre: 'Rusia', paraiso_fiscal: false})
```

Y a partir de estos datos se establecerán las siguientes relaciones:

```
MERGE (pe1)-[:TITULAR]->(c1)
MERGE (pe1)-[:TITULAR]->(c2)
MERGE (pe2)-[:TITULAR]->(c3)
MERGE (pe2)-[:TITULAR]->(c4)
MERGE (pe3)-[:TITULAR]->(c5)

MERGE (c1)-[:TRANSFIERE {importe: 50000, fecha: "2024-11-01"}]->(c2)
MERGE (c2)-[:TRANSFIERE {importe: 20000, fecha: "2024-12-12"}]->(c4)
MERGE (c4)-[:TRANSFIERE {importe: 5000, fecha: "2025-01-05"}]->(c3)
MERGE (c3)-[:TRANSFIERE {importe: 300, fecha: "2025-01-029"}]->(c5)
MERGE (c3)-[:TRANSFIERE {importe: 100, fecha: "2025-01-029"}]->(c1)
MERGE (c3)-[:TRANSFIERE {importe: 700, fecha: "2025-01-029"}]->(c2)

MERGE (pe1)-[:USA]->(md1)
MERGE (pe2)-[:USA]->(md1)
MERGE (pe3)-[:USA]->(md2)

MERGE (c1)-[:REGISTRADA]->(pa2)
MERGE (c2)-[:REGISTRADA]->(pa1)
```

```
MERGE (c3)-[:REGISTRADA]->(pa2)
MERGE (c4)-[:REGISTRADA]->(pa1)
MERGE (c5)-[:REGISTRADA]->(pa2)

MERGE (pe1)-[:RESIDE_EN]->(pa2)
MERGE (pe2)-[:RESIDE_EN]->(pa3)
MERGE (pe3)-[:RESIDE_EN]->(pa2)
```

En la base de datos se tienen tres personas, dos de ellas relacionadas con un mismo medio, pero viviendo en países diferentes. Se tienen 5 cuentas, alojadas en España y las Islas Caimán, entre las cuales se hacen una serie de transferencias con diferentes cantidades.

A partir de estos datos y relaciones podemos obtener 5 tipos de consultas:

1. Cuantas con altos balances que se encuentren en paraísos fiscales.

```
MATCH (c:Cuenta)-[:REGISTRADA]->(p:Pais {paraiso_fiscal: true})
MATCH (pr:Persona)-[:TITULAR]->(c)
WHERE c.balance > 10000
RETURN c.iban, c.balance, p.nombre, pr.nombre
ORDER BY c.balance DESC;
```

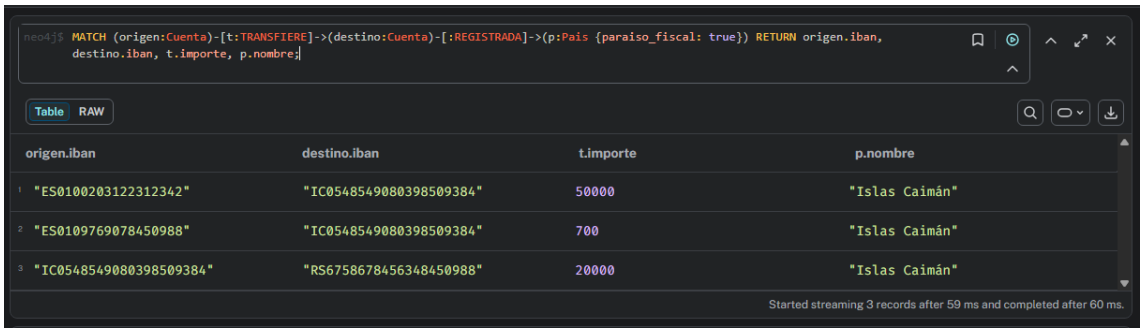
En este caso buscaremos las cuentas con un balance considerable (más de 100 mil euros) en algún paraíso fiscal.



2. Detectar aquel flujo de dinero que pasa por países considerados países fiscales.

```
MATCH (origen:Cuenta)-[t:TRANSFIERE]->(destino:Cuenta)-
[:REGISTRADA]->(p:Pais {paraiso_fiscal: true}) RETURN
origen.iban, destino.iban, t.importe, p.nombre;
```

En este caso buscamos las transferencias hechas desde cualquier cuenta a cuentas que se encuentren en paraísos fiscales, con el objetivo de detectar la cantidad de dinero que se está moviendo.



3. Identificar personas que puedan tener cierta relación debido a que usen los mismos medios.

```
MATCH (p1:Persona)-[:USA]->(md:Medio) MATCH (p2:Persona)-[:USA]->(md) WHERE p1 <> p2 WITH COLLECT(DISTINCT p2.nombre) as personas, md, p1 RETURN p1.nombre, personas, md.valor AS medio_compartido;
```

Aquí buscaremos que personas comparten medios usados para realizar pagos, transacciones u otros movimientos, de forma que podremos ver las posibles relaciones existentes que no conocíamos.



4. Toda la información de una determinada persona.

```
MATCH (pe:Persona {dni: "12345678A"}) OPTIONAL MATCH (pe)-[:TITULAR]->(c:Cuenta)-[:REGISTRADA]->(pais:Pais) OPTIONAL MATCH (pe)-[:USA]->(md:Medio) RETURN pe.nombre, pe.dni, collect(DISTINCT c.iban) AS cuentas, collect(DISTINCT pais.nombre) AS paises, collect(DISTINCT md.valor) AS medios;
```

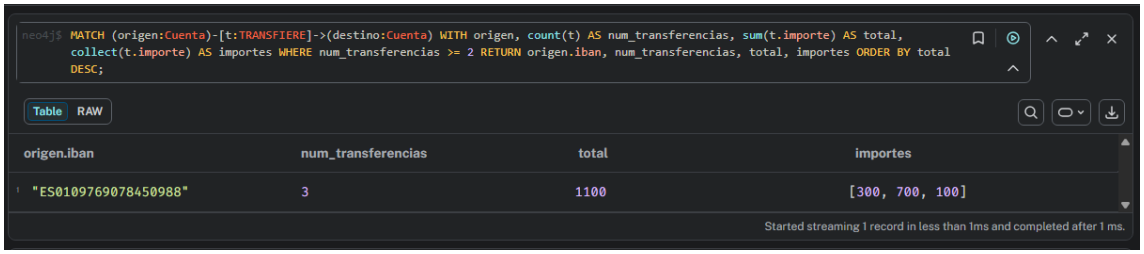
Esta consulta es útil para saber toda la información de una determinada persona: desde su información básica, hasta los diferentes países donde tiene cuentas.



5. Detectar transferencias de importes pequeños a una misma cuenta de destino desde la misma cuenta de origen.

```
MATCH (origen:Cuenta)-[t:TRANSFIERE]->(destino:Cuenta)
WITH origen, count(t) AS num_transferencias, sum(t.importe) AS
total, collect(t.importe) AS importes
WHERE num_transferencias >= 2
RETURN origen.iban, num_transferencias, total, importes
ORDER BY total DESC;
```

Esta consulta es muy útil debido a que se puede localizar grandes cantidades movidas desde una cuenta, pero con movimientos pequeños. De esta forma se puede comprobar si una cuenta está moviendo una cantidad importante de dinero sin que sea salten las alarmas al hacer una transferencia muy grande.



ANEXO. Captura de pantalla

Ejercicio 3.1.

